ОТЧЕТ

по лабораторной работе №4
по курсу «Логика»
на тему «Бинарное дерево поиска.»

Выполнили:
студенты группы 25ВВВ4
Волковец А.С
Пантюхин З.С

Приняли:
к.т.н. Юрова О.В.
к.т.н. Деев М.В.

1. # Цель работы

Научиться работать с бинарными деревьями

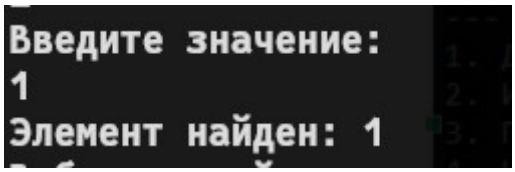
2. # Лабораторное задание

Задание

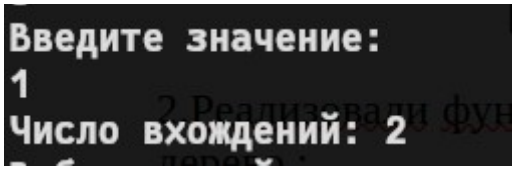
1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

3. # Ход работы

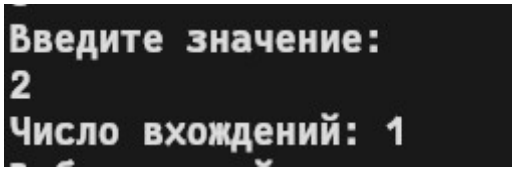
1. Реализовали алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.:



2.Реализовали функцию подсчёта числа вхождений заданного элемента в дерево.:



3.Изменили функцию добавления элементов для исключения добавления одинаковых символов.:



2. Листинг

Файл main.cpp

```
#include <iostream>
#include <limits>
using namespace std;
struct Node {
    int data;
    int count;
    Node* right;
    Node* left;
```

```
    Node(int val) : data(val), count(1), left(nullptr), right(nullptr) {}
};

class BinarySearchTree {
private:
    Node* root;

    Node* insert(Node* node, int val) {
        if (node == nullptr) {
            return new Node(val);
        }

         if (val == node->data) { /*ЕСЛИ УБРАТЬ ЭТО УСЛОВИЕ, ТО ДУБЛИКАТ
ПРОИГНОРИРУЕТСЯ И ДОБАВЛЕНИЕ ОДИНАКОВЫХ ЭЛЕМЕНТОВ БУДЕТ ИСКЛЮЧЕНО*/
            node->count++;
        }

        else if (val < node->data) {
            node->left = insert(node->left, val);
        }
        else {
            node->right = insert(node->right, val);
        }
        return node;
    }


    void tree(Node* node) {
        if (node == nullptr) {
            return;
        }
        tree(node->left);
        std::cout << node->data << " ";
        tree(node->right);
    }
    void getcount(Node* node, int val) {
        if (node == nullptr){
            cout << "Элемент не найден" << endl;
            return;
        }
        if (node->data == val) {
            cout << "Число вхождений: " << node->count << endl;
            return;
        }
        if (node->data < val) {
            return getcount(node->right, val);
        }
        else {
            return getcount(node->left, val);
        }
    }
    void find(Node* node, int val) {
        if (node == nullptr){
            cout << "Элемент не найден" << endl;
            return;
        }
        if (node->data == val) {
            cout << "Элемент найден: " << node->data << endl;
```

```
            return;
        }
        else if (node->data < val) {
            return find(node->right, val);
        }
        else {
            return find(node->left, val);
        }
    }
public:
    BinarySearchTree() : root(nullptr) {}
    void insert(int val) {
        root = insert(root, val);
    }

    void print_tree() {
        if (root == nullptr) {
            cout << "Дерево пустое" << endl;
        }
        tree(root);
        std::cout << std::endl;
    }
    void find(int val) {
        return find(root, val);
    }
    void getcount(int val){
        return getcount(root, val);
    }
};

int main() {
    BinarySearchTree bst;

    int choice = 0;
    while (choice != 5) {
        std::cout << "Выберете действие:\n"<< std::endl;
        std::cout << "1: Добавить элемент\n" << std::endl;
        std::cout << "2: Найти элемент\n" << std::endl;
        std::cout << "3: Найти число вхождений элемента\n" << std::endl;
        std::cout << "4: Список элементов\n" << std::endl;
        std::cout << "5: Выход\n" << std::endl;
        if (!(std::cin >> choice)) {
              std::cout << "Введено не число\n";
                    std::cin.clear();
                    std::cin.ignore(std::numeric_limits<std::streamsize>:
:max(), '\n');
        }
        if (choice == 1) {
            int val;
            cout  << "Введите значение: " << endl;
            cin >> val;
            bst.insert(val);
        }
        if (choice == 2) {
            int val;
            cout  << "Введите значение: " << endl;
            cin >> val;
            bst.find(val);
```

```
        }
        if (choice == 3) {
            int val;
            cout  << "Введите значение: " << endl;
            cin >> val;
            bst.getcount(val);
        }
        if (choice == 4) {
            bst.print_tree();
        }
    }
    return 0;

}
```